Knight Connection

AlgoExpert · Hard · Arrays

The problem

- Two knights on an infinite board move alternately/together; minimum number of turns for them to meet = ceil(knight-move distance / 2).
- BFS from one knight to the other.

Intuition

- Knights cover two cells per joint turn, so compute the minimum knight moves to close the gap, then halve it rounding up.

Brute force vs optimal

- Move both knights simultaneously → state space squared
- BFS from one knight to the other, halve (round up) → $O(d^2)$ cells
- Infinite board: no bounds, use a seen set

Approach

- 1. BFS knight moves from the origin to the offset between knights
- 2. Count moves needed
- 3. Answer is ceiling of moves over two

Worked example

```
BFS level 0: 1 squares

e.g. [(0, 0)]
```

Combined moves 0: 1 positions

Worked example

```
BFS level 1: 8 squares

e.g. [(1, 2), (2, 1), (-1, 2), (-2, 1), (1, -2), (2, -1)]
```

Combined moves 1: 8 positions

Worked example

```
BFS level 2: 8 squares

e.g. [(2, 4), (3, 3), (0, 4), (-1, 3), (2, 0), (3, 1)]
```

Combined moves 2: 8 positions

Worked example

```
reached (4, 2) after 2 total moves

two knights meet halfway → ⌈2/2⌉ = 1 turns
```

Answer: 1

Java solution

```java
public class KnightConnection {
    public static int knightConnection(int[] a, int[] b) {
        int[][] mv = {{1, 2}, {2, 1}, {-1, 2}, {-2, 1}, {1, -2}, {2, -1}, {-1, -2}, {-2, -1}};
        ArrayDeque<int[]> q = new ArrayDeque<>();
        Set<Long> seen = new HashSet<>();
        q.add(new int[]{a[0], a[1], 0});
        seen.add(key(a[0], a[1]));
        while (true) {
            int[] c = q.poll();
            if (c[0] == b[0] && c[1] == b[1]) return (c[2] + 1) / 2;
            for (int[] m : mv) {
                int x = c[0] + m[0], y = c[1] + m[1];
                if (seen.add(key(x, y))) q.add(new int[]{x, y, c[2] + 1});
            }
        }
    }

    private static long key(int x, int y) { return ((long) x << 32) ^ (y & 0xffffffffL); }
}
```

Complexity

- Time: O(d^2) for distance d
- Space: O(d^2)

Edge cases & pitfalls

- Round up odd distances
- Use a seen set (infinite board)
- Same square → 0

Interview tips & follow-ups

- Mention bidirectional BFS

Recap

- Knights cover two cells per joint turn, so compute the minimum knight moves to close the gap, then halve it rounding up.
- Complexity: O(d^2) for distance d